

Week-8 (DAY-3) – Handson – Keerthivasan R

Problem Statement

Traditional monolithic applications combine all business functionalities such as user management, product management, orders, payments, and reporting into a single codebase. As the application grows, this architecture leads to:

- Difficulty in scaling individual modules
- Slow deployment cycles
- Tight coupling between components
- Reduced fault isolation (failure in one module affects entire system)
- Harder maintenance and testing
- Limited technology flexibility

Modern businesses require scalable, flexible, and independently deployable systems to handle increasing users and complex business operations.

To address these challenges, the application must be redesigned using a **Microservices Architecture**.

Project Objective

Design and develop a microservices-based system using ASP.NET Core where:

- Each business module is developed as an independent service
 - Services communicate via REST APIs
 - Each service has its own database
 - Authentication is handled centrally using JWT
 - Services can be deployed and scaled independently
-

Proposed Microservices

Example system (E-Commerce or Enterprise Management):

1. **User Service**
 - Registration & Login
 - JWT Token Generation
2. **Product Service**
 - Add, Update, Delete Products
 - Product Listing
3. **Order Service**
 - Place Orders
 - Track Order Status
4. **Payment Service**
 - Handle Payments
 - Transaction Records
5. **API Gateway**
 - Single entry point for all services
 - Routes requests to appropriate microservices

Functional Requirements

- Independent services with separate databases
 - Inter-service communication using HTTP
 - Secure APIs using JWT Authentication
 - Proper error handling and logging
 - API Gateway implementation
-

Non-Functional Requirements

- Scalability
 - Fault isolation
 - Independent deployment
 - High availability
 - Security
-

Expected Outcome

The final system should:

- Allow services to run independently
 - Scale specific services based on load
 - Ensure secure communication between services
 - Reduce impact of failure in one service
 - Improve maintainability and performance
-

CODE:

 MicroservicesSolution/ appSettings.json

```
{
  "Jwt": {
    "Key": "MICROSERVICES_SUPER_SECRET_KEY_1234567890",
    "Issuer": "UserService",
    "Audience": "Microservices"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

MicroservicesSolution/ Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// Controllers
builder.Services.AddControllers();

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.AddSecurityDefinition("Bearer", new Microsoft.OpenApi.Models.OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = Microsoft.OpenApi.Models.SecuritySchemeType.Http,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = Microsoft.OpenApi.Models.ParameterLocation.Header
    });

    c.AddSecurityRequirement(new Microsoft.OpenApi.Models.OpenApiSecurityRequirement
    {
        {
            new Microsoft.OpenApi.Models.OpenApiSecurityScheme
            {
                Reference = new Microsoft.OpenApi.Models.OpenApiReference
                {
                    Type = Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            new string[] {}
        }
    });
});

// JWT Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,

            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!)
            )
        };
    });

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();

app.UseAuthentication(); // MUST be before Authorization
app.UseAuthorization();

app.MapControllers();
```

```
app.Run();
```

MicroservicesSolution/ Controllers/ ProductController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace ProductService.Controllers
{
    [Authorize]
    [ApiController]
    [Route("api/products")]
    public class ProductController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetProducts()
        {
            var products = new[]
            {
                new { Id = 1, Name = "Laptop", Price = 80000 },
                new { Id = 2, Name = "Mobile", Price = 30000 }
            };

            return Ok(products);
        }
    }
}
```

MicroservicesSolution/ Userservice/ Controllers/ AuthController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using UserService.Data;
using UserService.Models;
using System.Linq;

namespace UserService.Controllers
{
    [ApiController]
    [Route("api/auth")]
    public class AuthController : ControllerBase
    {
        private readonly UserDbContext _context;
        private readonly IConfiguration _configuration;

        public AuthController(UserDbContext context, IConfiguration configuration)
        {
            _context = context;
            _configuration = configuration;
        }

        // REGISTER
        [HttpPost("register")]
        public IActionResult Register(User user)
        {
            _context.Users.Add(user);
            _context.SaveChanges();
            return Ok("User registered successfully");
        }

        // LOGIN (JWT)
        [HttpPost("login")]
        public IActionResult Login(User user)
        {
            var existingUser = _context.Users
```

```

        .FirstOrDefault(u => u.Username == user.Username
            && u.Password == user.Password);

    if (existingUser == null)
        return Unauthorized("Invalid username or password");

    var claims = new[]
    {
        new Claim(ClaimTypes.Name, existingUser.Username)
    };

    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]));

    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _configuration["Jwt:Issuer"],
        audience: _configuration["Jwt:Audience"],
        claims: claims,
        expires: DateTime.Now.AddMinutes(30),
        signingCredentials: creds
    );

    return Ok(new
    {
        token = new JwtSecurityTokenHandler().WriteToken(token)
    });
}
}
}

```

📁 MicroservicesSolution/ Userservice/ Data/ UserDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using UserService.Models;

namespace UserService.Data
{
    public class UserDbContext : DbContext
    {
        public UserDbContext(DbContextOptions<UserDbContext> options)
            : base(options)
        {
        }

        public DbSet<User> Users { get; set; }
    }
}

```

📁 MicroservicesSolution/ Userservice/ Models/ users.cs

```

namespace UserService.Models
{
    public class User
    {
        public int Id { get; set; }
        public string Username { get; set; }
        public string Password { get; set; }
    }
}

```

📁 MicroservicesSolution/ Userservice/ appSettings.json

```

{
  "Jwt": {
    "Key": "MICROSERVICES_SUPER_SECRET_KEY_1234567890",
    "Issuer": "UserService",
    "Audience": "Microservices"
  },

```

```

"ConnectionStrings": {
  "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=UserServiceDb;Trusted_Connection=True;"
},

"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
  }
},

"AllowedHosts": "*"
}

```

 MicroservicesSolution/ Userservice/ Program.cs

```

using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using System.Text;
using UserService.Data;

var builder = WebApplication.CreateBuilder(args);

// Controllers
builder.Services.AddControllers();

// DbContext
builder.Services.AddDbContext<UserDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// JWT Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,

            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
        };
    });

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Middleware
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthentication(); // 📁 IMPORTANT
app.UseAuthorization();

app.MapControllers();

app.Run();

```

OUTPUT:

 **Swagger**
Powered by SMARTBEAR

Select a definition **UserService v1**

UserService

1.0 OAS 3.0
<https://localhost:7053/swagger/v1/swagger.json>

Auth

POST /api/auth/register

Parameters

No parameters

Request body

application/json

```
{
  "username": "admin",
  "password": "1234"
}
```

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7053/api/auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "admin",
    "password": "1234"
  }'
```

Request URL

```
https://localhost:7053/api/auth/register
```

Server response

Code	Details
200	Response body User registered successfully Download Response headers content-type: text/plain; charset=utf-8 date: Tue, 14 Apr 2026 05:53:50 GMT server: Kestrel

POST /api/auth/login

Parameters

No parameters

Request body

application/json

```
{
  "username": "admin",
  "password": "1234"
}
```

```
curl -X 'POST' \
  https://localhost:7053/api/auth/login \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "admin",
    "password": "1234"
  }'
```

```
https://localhost:7053/api/auth/login
```

Code	Details
------	---------

Response body

[illegible]

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 14 Apr 2016 05:03:59 GMT
server: Kestrel
```

Product

```
GET /api/products
```

Parameters

No parameters

Execute

Responses

[illegible]

Request URL

```
https://localhost:7283/api/products
```

Server response

```
https://localhost:7263/api/products
```

Server response

Code	Details
------	---------

200

Response body

```
{
  "id": 1,
  "name": "Laptop",
  "price": 50000
},
{
  "id": 2,
  "name": "Mobile",
  "price": 5000
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 14 Apr 2026 05:55:46 GMT
server: gsestrel
```

Responses

Code	Description
------	-------------

200

OK

Links

Mr. Baker

